

Evolved Agentic Engineering (EAE) Methodology

1. Core Philosophy

Evolved Agentic Engineering (EAE) represents the evolution of AI-assisted software development. It transitions workflows from casual, conversational scripting (Vibe Coding) and manual staging (Classic ICM) to autonomous, sandboxed software factories. EAE treats Large Language Models as deterministic compilation targets governed by strict verification pipelines.

2. The Three-Tier Prompting Hierarchy

EAE partitions system context across three distinct operational layers to maximize precision, enforce security boundaries, and minimize context rot:

- **Tier 1: Global Identity:** System-level custom instructions (e.g. AGENTS.md) that apply safety baselines, identity roles, and non-negotiable rules (like file backup requirements) across all projects.
- **Tier 2: Repository Context:** Codebase-specific instructions located in AGENTS.md and CONTEXT.md. This layer documents local file conventions, library versions, schema maps, and project expectations.
- **Tier 3: Execution Prompts:** Task-specific instructions comprised of conversational Thin Prompts and pre-packaged, trigger-based workflow templates (skills/ directory).

Evolved Agentic Engineering (EAE) Methodology

3. Dynamic Memory & Self-Correction

To combat retrieval amnesia in long-running projects, EAE implements a local, dynamic learning loop. Instead of resetting past learnings, agents actively write compilation, linting, and runtime anomalies back to a repository decision log (MEMORY.md). This builds a compounding set of constraints that prevents the AI from falling into repetitive code errors (the Vibe Coding Doom Loop).

4. Continuous Validation & Sandboxing

Under EAE, generated code is treated as fundamentally flawed until validated. Using Test-Driven AI (TDAI), the manager agent spawns worker agents who write tests first and run test suites (pytest/linters) inside secure local sandboxes. The AI iterates on code modifications autonomously until all tests output a clean pass.

5. Multi-Agent Orchestration

Complexity is managed by splitting workflows across specialized, sequential roles (Architects, Developers, QA/Reviewers). Handoffs occur asynchronously via version-controlled markdown artifacts (like plan.md) rather than broad chat windows, keeping agent context windows highly focused.

6. Application Suitability

- **Where it Excels:** Complex, multi-file software engineering tasks, large monorepos, and automated continuous-integration workflows.
- **Where to Avoid:** Trivial, single-file scripts, freeform brainstorming, or prototyping where speed overrides system safety.